



EÖTVÖS LORÁND TUDOMÁNYEGYETEM

INFORMATIKAI KAR

PROGRAMOZÁSI NYELVEK ÉS FORDÍTÓPROGRAMOK TANSZÉK

Smart Turntable Assistant

Témavezető:

Bense Viktor

Doktorandusz

Szerző:

Kiss Dániel

programtervező informatikus BSc

Budapest, 2026

Tartalomjegyzék

1. Bevezetés	3
1.1. Témaválasztás indoklása	3
1.2. A megoldandó feladat leírása	4
1.3. A szakdolgozat célkitűzései	4
2. Felhasználói dokumentáció	6
2.1. A megoldott probléma megfogalmazása	6
2.2. A felhasznált módszerek rövid leírása	6
2.3. A program használatához szükséges információk	7
2.3.1. Rendszerkövetelmények és Előfeltételek	7
2.3.2. Telepítés és Indítás lépésről lépésre	8
2.3.3. A Felhasználói Felület (UI) felépítése és használata	8
3. Fejlesztői dokumentáció	15
3.1. Követelményelemzés és specifikáció	15
3.1.1. Funkcionális követelmények	15
3.1.2. Nem-funkcionális követelmények	16
3.2. Szoftver architektúra és szerkezet	16
3.2.1. Adatbázis Modell	17
3.2.2. Konkurencia és szálkezelés	17
3.3. Megvalósítási részletek (Backend modulok)	19
3.3.1. Digitális jelfeldolgozás (DSP) – <code>processing.py</code>	19
3.3.2. Zenefelismerés (Recognition) – <code>recognition_service.py</code>	21
3.3.3. Metaadat-vízesés (Waterfall) – <code>metadata_service.py</code>	21
3.3.4. Háttérfolyamatok és Szálkezelés – <code>tasks.py</code>	22
3.4. Tesztelés és Értékelés	23
3.4.1. Automatizált Egységtesztelés (Unit Testing)	23
3.4.2. Frontend renderelés, Vizuális Megjelenítés és Témázás	25

3.4.3. Kliens-oldali Logika és Állapotkezelés (useDashboard.js) . . .	27
3.4.4. Profilkezelés és Hardver Konfiguráció (useProfile.js)	29
3.4.5. Manuális és integrációs tesztelés	30
3.4.6. Teljesítmény-optimalizáció	30
3.5. Mesterséges intelligencia alkalmazása a fejlesztés során	30
4. Összegzés és jövőbeli fejlesztések	32
4.1. Az elért eredmények értékelése	32
4.2. Jövőbeli fejlesztési lehetőségek	32
Ábrajegyzék	34
Ábrajegyzék	35
Táblázatjegyzék	36
Algoritmusjegyzék	37
Forráskódjegyzék	38

1. fejezet

Bevezetés

1.1. Témaválasztás indoklása

Az elmúlt években a zeneipar az analóg fizikai hordozók, különösen a bakelit-lemezek népszerűségének példátlan visszatérését tapasztalta. Az audiofileket és a hétköznapi embereket egyaránt vonzza a mechanikus élmény, a szándékos cselekvés, amellyel egy albumot az elejétől a végéig meghallgatnak, valamint az analóg hangzás egyedi, meleg akusztikai jellemzője. Ugyanakkor egy olyan korszakban, amelyet a digitális streaming platformok, mint a Spotify és az Apple Music uralnak, a hagyományos hanglemez hallgatási élmény alapvetően elszigetelt marad a modern digitális ökoszisztémától.

A témát azért választottam, mert létezik egy jelentős technológiai szakadék az analóg audio jel és a modern metaadat-kezelés között. Egy olyan eszköz létrehozása, amely képes hidat képezni a fizikai lemezjátszó és a digitális kényelmi funkciók között, nemcsak technikai kihívást jelent a valós idejű jelfeldolgozás terén, hanem valódi hozzáadott értéket nyújt a zenehallgatók számára, akik nem szeretnék lemondni az analóg élményről, de igényelnék a digitális kor előnyeit.

Ez a szakdolgozat egy Smart Turntable Assistant (Okos Lemezjátszó Asszisztens) tervezését, architektúráját és megvalósítását mutatja be, amely egy szoftvervezérelt hidat képez az analóg audio hardverek és a modern digitális kényelmi funkciók között. Valós idejű jelfeldolgozás és külső API integrációk felhasználásával ez a projekt arra törekszik, hogy modernizálja az analóg zenehallgatás élményét anélkül, hogy a bakelit-lemez-hallgatás élményébe, hangvilágába beleszólna.

1.2. A megoldandó feladat leírása

A projekt célja egy olyan „Okos Lemezjátszó Asszisztens” szoftveres rendszer megvalósítása, amely passzívan figyeli egy szabványos lemezjátszó audio kimenetét, és automatikusan nyújt digitálisan kiegészítéseket a hallgatáshoz.

A rendszernek a következő alapvető feladatokat kell megoldania:

- **Automatikus felismerés:** A bejövő analóg jel alapján meg kell határozni, hogy egy zeneszám elkezdődött-e, majd audio fingerprinting segítségével azonosítani kell a pontos előadót és dalcímet.
- **Metaadat-szinkronizáció:** A felismerést követően a rendszernek le kell kérdeznie a számhoz tartozó albumborítót és a szinkronizált dalszöveget, majd azt valós időben meg kell jelenítenie a felhasználónak.
- **Hardveres monitorozás:** A szoftvernek matematikai elemzéssel nyomon kell követnie a hangszedő (tű) kopását a lejátszási órák alapján, valamint diagnosztikai adatokat (pl. motorrezgés, felületi zaj) kell szolgáltatnia.
- **Digitális naplózás:** A hallgatási élményt automatikusan szinkronizálnia kell külső platformokkal (pl. Last.fm), így az analóg lejátszás is digitális nyomon követhetővé válik.

1.3. A szakdolgozat célkitűzései

A szakdolgozat elsődleges célja egy robusztus, valós idejű Kliens-Szerver alkalmazás megtervezése és megvalósítása, amely passzívan figyeli egy szabványos lemezjátszó audio kimenetét, és „okos” funkciók átfogó készletét nyújtja. A rendszert úgy terveztem, hogy egy USB audio interfészhez csatlakoztatott, alacsony erőforrás-igényű hardveren fusson.

A fent vázolt problémák megoldása érdekében a projekt a következő műszaki célkitűzéseket határozza meg:

- **Valós idejű jelfeldolgozás:** Egy konkurens Python backend fejlesztése, amely képes az analóg audio stream folyamatos elemzésére egy zeneszám kezdetének vagy végének detektálásához, valamint a hardverdiagnosztikai adatok matematikai kiszámításához.

- **Audio Fingerprinting és metaadat-bővítés:** Egy robusztus felismerési pipeline implementálása, amely rövid hangmintákat rögzít, és audio fingerprinting szolgáltatások segítségével azonosítja azokat. Továbbá egy waterfall elvű metaadat-lekérdező rendszer tervezése, amely megbízhatóan lekéri a pontos sávhosszokat, albumneveket és nagy felbontású borítóképeket.
- **Automatizált digitális szinkronizáció:** Háttér szolgáltatások létrehozása az időbélyeggel ellátott dalszövegek letöltésére, valamint egy automatizált Last.fm scrobbling funkció implementálása, amely pontosan tiszteletben tartja a sáv eltelt lejátszási idejét.
- **Adatbázis- és hardverkezelés:** Egy relációs adatbázis megvalósítása a hallgatási előzmények tárolására, a testreszabott vizuális preferenciák mentésére, valamint a felhasználó által definiált lemezjátszó-hangszedők életciklusának pontos nyomon követésére.
- **Modern, reaktív felhasználói felület:** Egy reszponzív Single Page Application (SPA) tervezése Vue.js használatával, amely WebSockets technológián keresztül kommunikál a backenddel. Az interfésznek zero-latency vizuális visszajelzést kell nyújtania a felhasználók számára, és dinamikus tematikus testreszabhatóságot kell kínálnia.

2. fejezet

Felhasználói dokumentáció

2.1. A megoldott probléma megfogalmazása

A hagyományos bakelit-lemezjátszó legnagyobb hátránya, hogy a generált jel egy nyers elektromos jel, amely nem tartalmaz metaadatokat. Ez azt jelenti, hogy a felhasználó számára nincs automatikus módja a lejátszott zeneszámok naplózására, nincs hozzáférése a szinkronizált dalszövegekhez, és nincs pontos eszköze a tű kopásának vagy a lemezjátszó technikai állapotának ellenőrzésére. A Smart Turntable Assistant ezen „információs vakpontot” törli el egy intelligens szoftverréteg bevezetésével.

2.2. A felhasznált módszerek rövid leírása

A rendszer egy kliens-szerver architektúrán alapul, ahol a backend és a frontend különálló egységekben működnek:

- **Audio elemzés és azonosítás:** A zeneszámok azonosításához külső Audio Fingerprinting API-kat (pl. Shazam) alkalmaz a rendszer, amelyek egyedi akusztikai „ujjlenyomatokat” hasonlítanak össze egy globális adatbázissal.
- **Folyamatos internetkapcsolat:** Fontos megjegyezni, hogy a program zavartalan működéséhez **aktív internetkapcsolat szükséges**. Ennek oka, hogy az analóg jelből kinyert „ujjlenyomatokat” a rendszer online szerverekkel azonosítja, illetve a nagy felbontású albumborítók, a szinkronizált dalszövegeket és a Last.fm naplózást is az interneten keresztül éri el. Internet nélkül a szoftver csak a helyi diagnosztikai adatokat képes mutatni.

- **Valós idejű kommunikáció:** A szerver és a felhasználói felület közötti adatcsere WebSockets technológiával történik, ami lehetővé teszi, hogy a diagnosztikai adatok és a dalszövegek késleltetés nélkül érkezzenek a képernyőre.
- **Felhasználói felület:** Egy modern, webes felület (SPA), amely egy böngészőn keresztül bármilyen eszközön (mobiltelefon, táblagép, asztali számítógép) elérhető.
- **Adatkezelés:** Az adatbázis helyben tárolja a felhasználói beállításokat és a tū élettartamát, így az adatok nem vesznek el az eszköz újraindításakor.

2.3. A program használatához szükséges információk

2.3.1. Rendszerkövetelmények és Előfeltételek

A Smart Turntable Assistant használatához az alábbi hardveres és szoftveres feltételeknek kell teljesülniük:

- **Hardver:**
 - Egy szabványos analóg lemezjátszó.
 - Egy USB audio interfész (hangkártya) vagy erősítő, amely összeköti a lemezjátszót a számítógéppel/szerverrel.
 - Egy számítógép, laptop vagy egykártyás számítógép (pl. Raspberry Pi), amelyen a szoftver fut.
- **Szoftver (Operációs rendszer):** A program Windows, macOS és Linux operációs rendszereken egyaránt futtatható.
- **Függőség (Dependency):** A telepítéshez elengedhetetlen a **Docker** és a **Docker Compose** nevű szoftverek megléte a gépen. Ezek biztosítják, hogy a program minden szükséges összetevővel együtt, egy zárt dobozban (konténerben) hibamentesen induljon el.
- **Böngésző:** Bármilyen modern webböngésző (Google Chrome, Mozilla Firefox, Safari, Edge).

2.3.2. Telepítés és Indítás lépésről lépésre

A szoftver telepítése és elindítása a következő egyszerű lépésekből áll:

1. **Docker telepítése:** Amennyiben még nincs a gépén, töltsse le és telepítse a Docker Desktop alkalmazást a hivatalos weboldáról, majd indítsa el.
2. **Fájlok előkészítése:** Töltsse le a Smart Turntable Assistant projekt mappáját a számítógépére, amely tartalmazza a `docker-compose.yml` nevű indítófájlt.
3. **Terminál megnyitása:** Nyisson meg egy parancssort (Windows alatt: Parancssor / PowerShell, Mac/Linux alatt: Terminal), és navigáljon abba a mappába, ahová a fájlokat letöltötte.
4. **A rendszer elindítása:** Gépelje be a következő parancsot, majd nyomja meg az Enter-t:

```
docker compose up -d
```

Ez a parancs letölti a szükséges összetevőket és a háttérben elindítja a programot. A `-d` kapcsoló biztosítja, hogy a terminált az indítás után be lehessen zárni.

5. **Felület megnyitása:** Nyissa meg a kedvenc webböngészőjét, és a címsorba gépelje be a következő címet: `http://localhost:5000` (Ha a szoftver egy másik gépen, pl. Raspberry Pi-n fut, a `localhost` helyére annak az eszköznek az IP címét kell beírni).

2.3.3. A Felhasználói Felület (UI) felépítése és használata

A webes felületet úgy terveztük meg, hogy letisztult és azonnal átlátható legyen. Az alkalmazás megnyitásakor egy bejelentkező képernyő (Login/Register) fogadja, ahol egy saját fiókot kell létrehoznia. Ez biztosítja, hogy a lemezejátszó-tű kopása és a beállításai csak az Ön számára legyenek elérhetőek.

Sikeres bejelentkezés után a képernyő bal oldalán a lejátszó, jobb oldalán pedig az információs fülek (menüpontok) helyezkednek el.

Főképernyő (Dashboard) és a Lejátszó panel

A bal oldali panel a rendszer „szíve”, amely a zenehallgatás vizuális élményét adja.

- **A lemezzátszó animációja:** Alapállapotban a digitális bakelitlemez áll. Amint a tűt a fizikai lemezre helyezi, a rendszer ezt érzékeli, és a képernyőn lévő lemez is forogni kezd („Listening...” felirat jelenik meg).
- **Az azonosított zeneszám:** Sikeres felismerés esetén a képernyőn megjelenik a lejátszott album nagy felbontású borítóképe, alatta pedig a zeneszám címe és az előadó. A borítókép a forgó lemez közepén (a címkén) is megjelenik.
- **Lemez színének megváltoztatása:** Ha jobb gombbal kattint (vagy hosszan rányom) a forgó bakelitlemezre, egy felugró menüből kiválaszthatja a lemez színét és stílusát, hogy az pontosan passzoljon a fizikai lemezéhez.
- **Kézi azonosítás (Detect Now):** Ha a rendszer valamiért nem ismerné fel automatikusan a dalt, a nagyítós „Detect Now” gombra kattintva bármikor kikényszeríthet egy új felismerési próbálkozást.

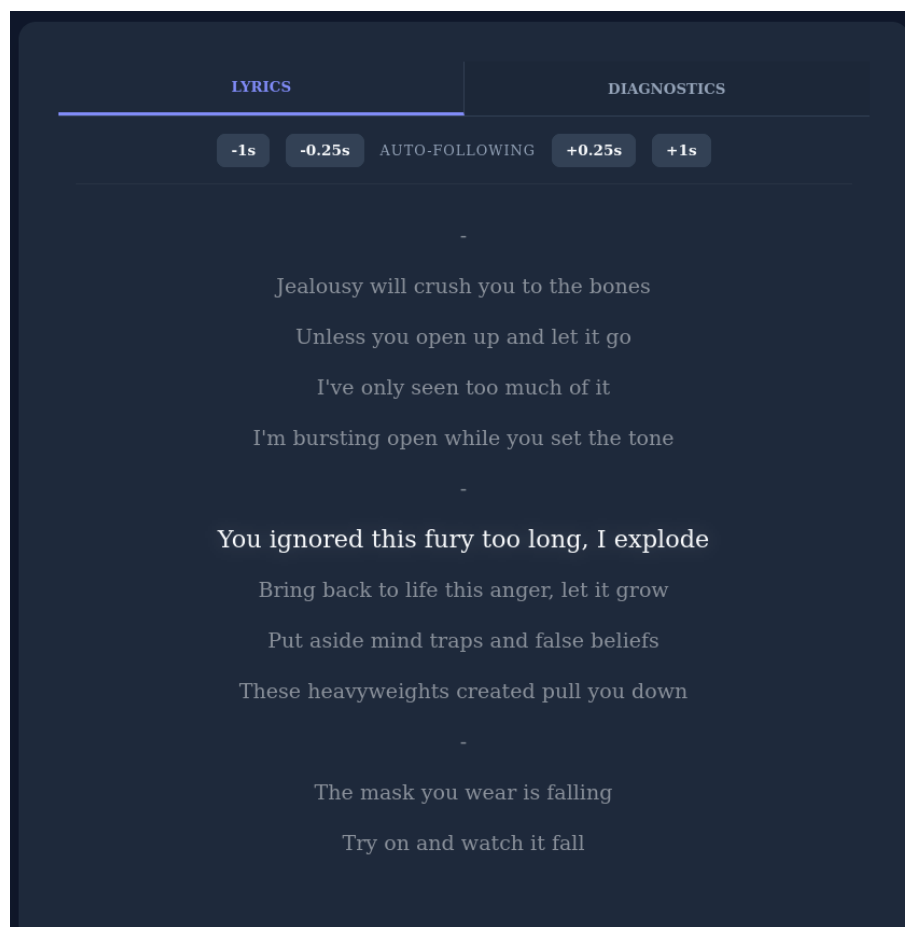


2.1. ábra. A Főképernyő lejátszó panelje aktív zenehallgatás közben.

Dalszöveg (Lyrics) fül

A jobb oldali panelen a „LYRICS” földre kattintva a felismert zeneszámhoz tartozó, időzített dalszöveg jelenik meg.

- **Automatikus görgetés:** A szöveg a zene ritmusában, automatikusan gördül, és az éppen énekelt sor kiemelve jelenik meg.
- **Finomhangolás (Sync controls):** Mivel a lemezjátszók sebessége (pitch) minimálisan eltérhet, előfordulhat, hogy a szöveg siet vagy késik. Ilyenkor a felül található gombokkal (-1s, -0.25s, +0.25s, +1s) tizedmásodperc pontos-sággal eltolhatja a szöveg időzítését.
- A **Resync** gombbal bármikor visszatérhet az automatikus követéshez, a bal oldali panelen lévő **Reset Sync** gombbal pedig alaphelyzetbe állíthatja az eltolásokat.



2.2. ábra. Szinkronizált dalszöveg megjelenítése és finomhangoló kezelőszervei.

Diagnosztika (Diagnostics) fül

Ezen a fülön valós idejű statisztikákat és mozgó grafikonokat talál, amelyek a hanglemez és a lemezjátszó technikai állapotát mutatják.

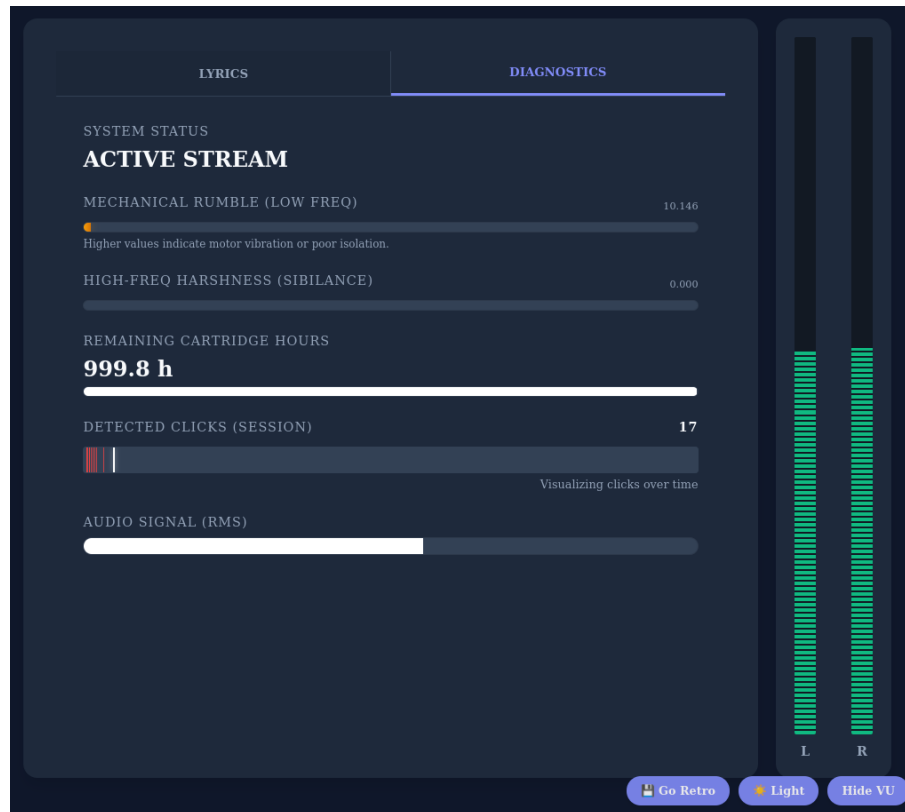
- **System Status (Rendszerállapot):** Mutatja, hogy a rendszer éppen várakozik (IDLE), aktív jelfolyamot vesz (ACTIVE STREAM), vagy a zene szünetel.
- **Rumble (Motorrezgés) és Sibilance (Torzítás):** Színes csúszkákkal (amelyek pirosra váltanak, ha az érték túl magas) jelzi a mélyfrekvenciás motorzajt és a magasfrekvenciás sercegést. Ezek a mutatók segítenek a lemezjátszó ideális beállításában.
- **Remaining Cartridge Hours (Hátralévő tű élettartam):** Egy folyamatjelző mutatja a beállított tű hátralévő üzemidejét.
- **Detected Clicks (Kattogás és Idővonal):** Nemcsak a pattogások összesített számát mutatja, hanem egy **kis idővonalon pöttyökkel jelzi**, hogy a zeneszám melyik másodpercében történt a sercegés. Ez segít azonosítani a lemez sérült részeit.
- **VU Meter (Kivezérlésjelző):** A hangerő szintjét mutatja. Ha a retro témát használja, ez klasszikus analóg mutatós műszerként (mutatós óraként) jelenik meg, ha modern témát, akkor LED-soros oszlopként.

Profil, Hardver Beállítások és Túkezelés

A jobb felső sarokban található avatarra (az Ön monogramjára) kattintva érheti el a privát profilját. Ez az oldal három fő részre van osztva:

1. Meghallgatott zenék és Hangszedők (Cartridges):

- **Listening History:** Itt láthatja a korábban sikeresen felismert és végighallgatott zeneszámait.
- **Hangszedő regisztrálása:** A „New” gombra kattintva adhatja meg új tűjének nevét és várható élettartamát (pl. 300 óra). A listában a lefelé mutató nyílra kattintva lenyithatja a tű beállításait, ahol egy kattintással aktívvá teheti („Activate”), módosíthatja a limitet, esetleg nullázhatja („Reset”) vagy törölheti a tűt.



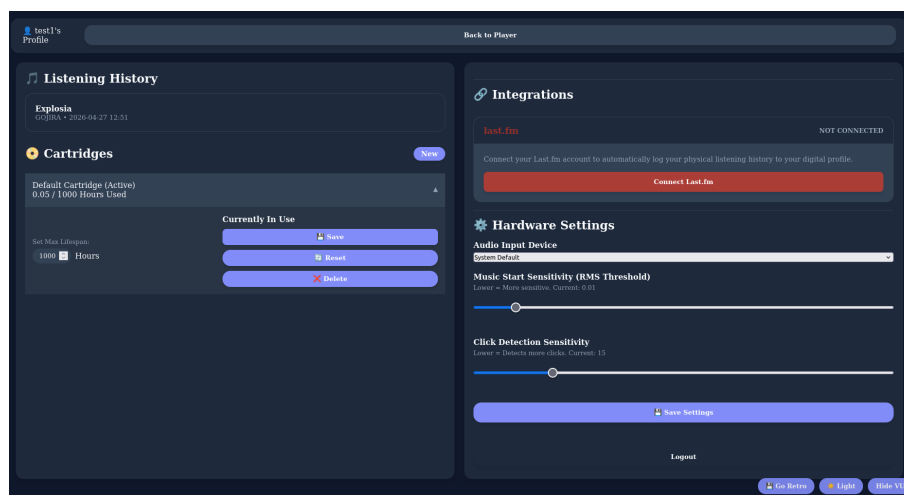
2.3. ábra. Hardveres diagnosztika, kivezérlésjelzők és a pattogások idővonala.

2. Integrációk (Last.fm): Ha szeretné, hogy a fizikai lemezeinek hallgatása is bekerüljön a digitális zenei statisztikáiba, itt a „Connect Last.fm” gombbal összekapcsolhatja a fiókját. A rendszer innentől automatikusan „scrobbolja” (naplózza) a felismert számokat.

3. Hardveres Beállítások (Hardware Settings): Ez egy kritikus menüpont a program helyes működéséhez:

- **Audio Input Device:** Itt választhatja ki a legördülő menüből, hogy melyik mikrofonon vagy hangkártyán keresztül érkezik a lemezjátszó hangja a gépbe.
- **Music Start Sensitivity:** Egy csúszka, amivel beállíthatja, hogy a rendszer milyen hangerősséget tekintsen a zeneszám kezdetének. Ha túl lassan ismeri fel az indulást, vegye kisebbre az értéket.
- **Click Detection Sensitivity:** Ezzel a csúszkával szabályozhatja, hogy mennyire legyen érzékeny a rendszer a lemez pattogására. Ha túl sok fals pattogást számol, tolja feljebb az értéket.

A változtatások érvényesítéséhez mindig kattintson a „Save Settings” gombra!



2.4. ábra. Profilkezelés, hangszedő konfiguráció és hardveres érzékenység beállítása.

3. fejezet

Fejlesztői dokumentáció

3.1. Követelményelemzés és specifikáció

A rendszer alapvető célja egy olyan szoftveres megoldás létrehozása, amely képes egy analóg audiojelből zenei metaadatokat és diagnosztikai információkat kinyerni.

3.1.1. Funkcionális követelmények

- **Valós idejű audio azonosítás:** A rendszernek folyamatosan figyelnie kell a bejövő audio streamet, és automatikusan detektálnia kell a lejátszás kezdetét. Az észlelés után rögzítenie kell egy rövid audio buffert, és külső audio fingerprinting API-kat kell lekérdeznie a sáv azonosításához.
- **Metaadat-bővítés:** Sikeres audio felismerés után másodlagos adatbázisokat kell lekérdeznie a nagy felbontású albumborítók, a pontos sáv hosszúság és a szabványosított albumnév lekéréséhez.
- **Hardverállapot-diagnosztika:** A szoftvernek matematikailag fel kell dolgoznia a hangot a hirtelen felületi pattogások (clicks) detektálására, az alacsony frekvenciás mechanikai motorrezgés (rumble) mérésére, és a magas frekvenciás sziszegő torzítás (sibilance) számszerűsítésére.
- **Hangszedő életciklus kezelés:** Adatbázis fenntartása a felhasználó lemezjátszó tűiről, nyomon követve az aktív hardver kumulatív lejátszási idejét.
- **Digitális hallgatási naplók:** Hitelesítés és automatikus scrobbling külső platformokra (pl. Last.fm).

3.1.2. Nem-funkcionális követelmények

- **Teljesítmény és késleltetés:** A háttérben futó audiofeldolgozó szálnak folyamatosan működnie kell anélkül, hogy blokkolná a webszerver műveleteit.
- **Megbízhatóság és Fallback mechanizmusok:** A hálózati lekérdezéseknek robusztus fallback stratégiákat kell alkalmazniuk. Ha egy elsődleges felismerő motor meghibásodik, a rendszernek meg kell próbálnia a másodlagos források használatát.
- **Adatbázis-konkurencia:** A relációs adatbázisnak támogatnia kell az egyidejű olvasási és írásai műveleteket (Write-Ahead Logging módban).

3.2. Szoftver architektúra és szerkezet

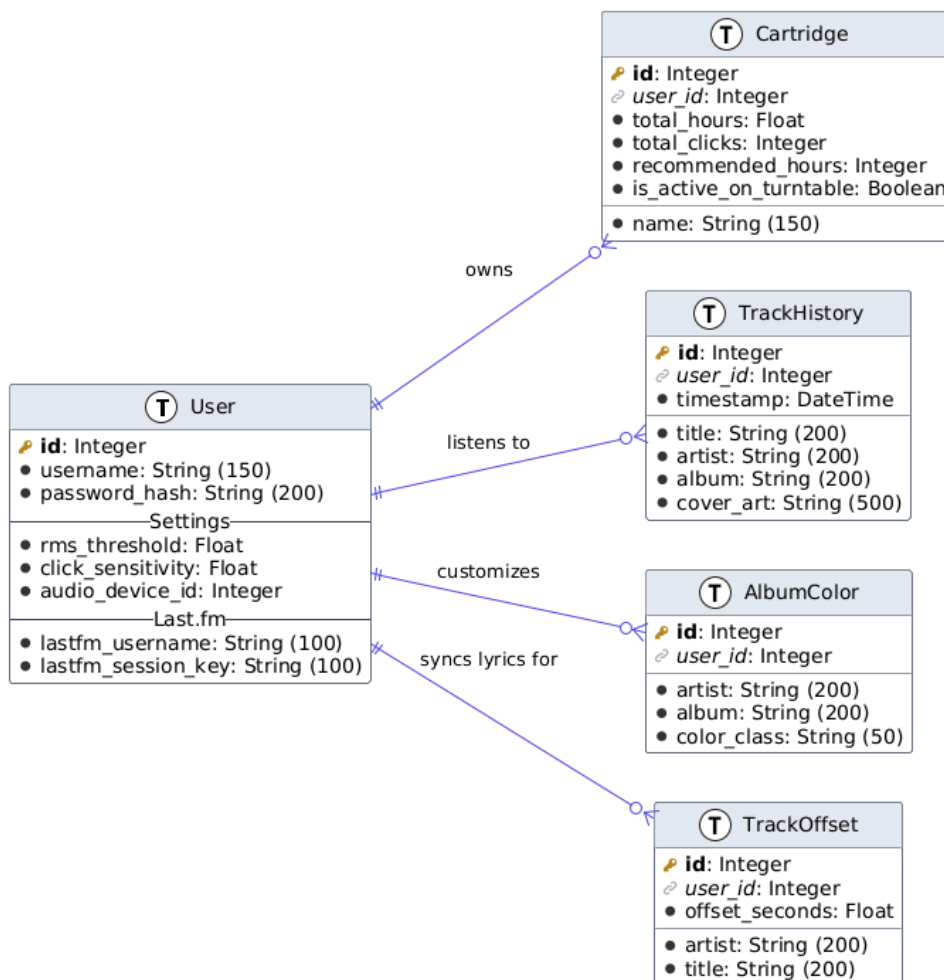
Az alkalmazás egy robusztus kliens-szerver modellt követ. A backend Python nyelven készült, mivel kiterjedt ökoszisztémája (NumPy [1], SciPy [2]) ideális a matematikai számításokhoz. A hang rögzítéséhez a `sounddevice` [3] könyvtárat alkalmazom. A webszerver réteget a `Flask` hajtja meg. A frontend és a backend közötti állapotmentes kommunikációt a Flask által biztosított RESTful API végpontok valósítják meg. A 3.1. táblázat foglalja össze az alkalmazás legfontosabb végpontjait és azok funkcióit.

Metódus	Végpont (URL)	Funkció leírása
POST	/auth/login	Felhasználó hitelesítése és a munkamenet (session) elindítása.
GET	/api/user/profile	A hitelesített felhasználó adatainak, hallgatási előzményeinek és regisztrált tűinek lekérése.
POST	/api/user/settings	Hardveres érzékenységi beállítások (RMS küszöb, kattogás érzékenység) mentése az adatbázisba.
POST	/api/cartridges/{id}/update_limits	Egy adott hangszedő (tű) várható maximális élettartamának módosítása.
POST	/api/cartridges/{id}/reset	Az aktív hangszedő eddigi kumulatív üzemóráinak nullázása tűcsere esetén.

3.1. táblázat. A rendszer alapvető REST API végpontjai és azok funkciói.

3.2.1. Adatbázis Modell

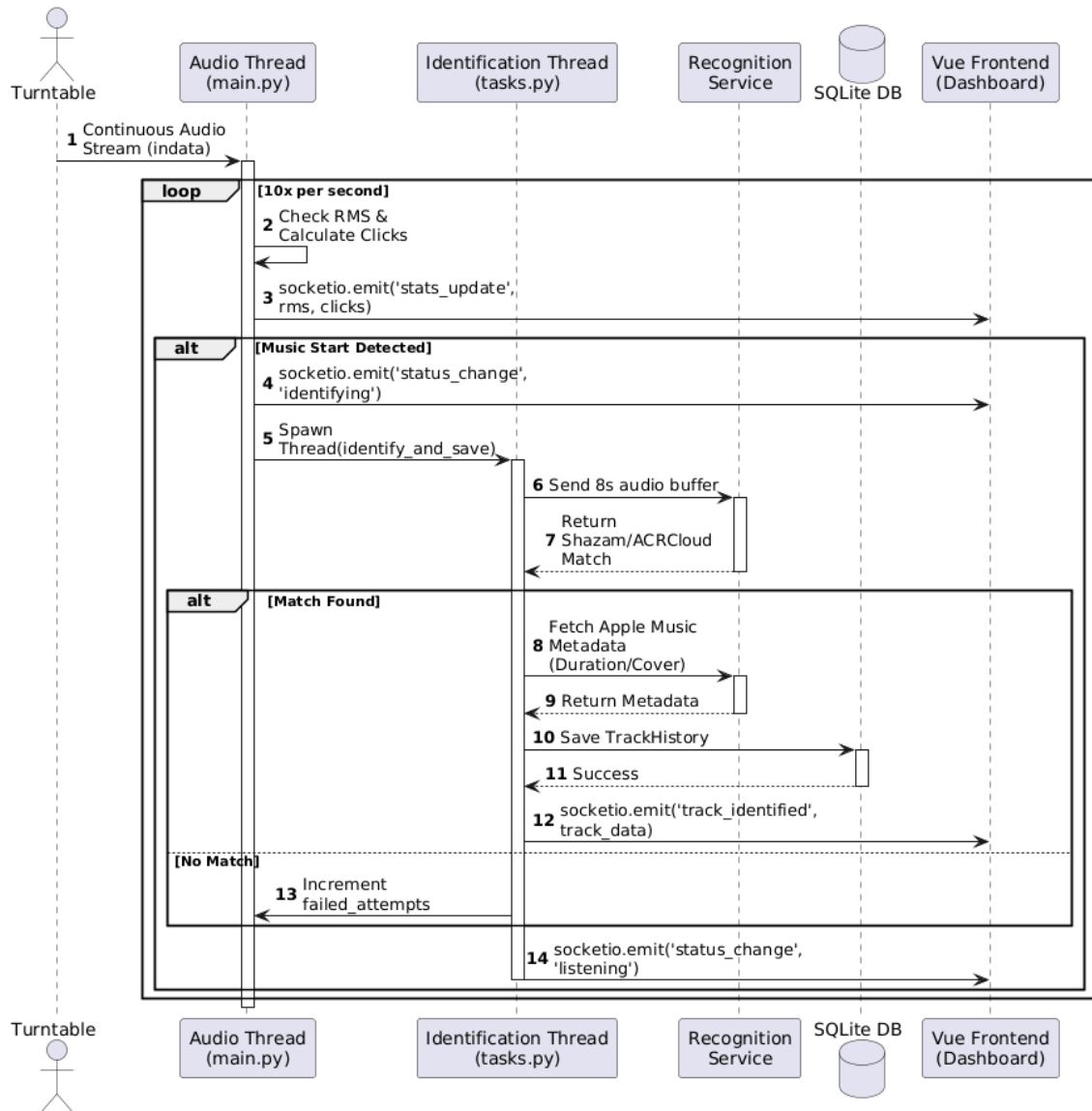
A perzisztens adattárolást SQLite kezeli SQLAlchemy ORM használatával (lásd 3.1. ábra). A `User` entitás biztosítja az adatok izolációját. A `Cartridge` tábla követi a tűk üzemóráit, míg a `TrackHistory` digitális hallgatási naplóként funkcionál.



3.1. ábra. Entity-Relationship Diagram az adatbázis sémájáról.

3.2.2. Konkurencia és szálkezelés

A backend többszálú architektúrát alkalmaz (lásd 3.2. ábra). Az elsődleges szál folyamatosan olvassa az audio puffert, számítja a diagnosztikát, és megállapítja egy új dal kezdetét. Ekkor elindít egy független másodlagos szálát, amely elvégzi a hálózati kéréseket az audio azonosításhoz, elkerülve az elemzés megfagyását.



3.2. ábra. UML Sequence Diagram a többszálú adatfeldolgozásról.

3.3. Megvalósítási részletek (Backend modulok)

A rendszer backendje több felelősségi körre van bontva. Az alábbiakban modulonként és metódusonként részletezem a megvalósítást és az alkalmazott matematikai modelleket.

3.3.1. Digitális jelfeldolgozás (DSP) – `processing.py`

Ez a modul felelős az analóg audiojel diszkrét matematikai elemzéséért. A `sounddevice` által rögzített többdimenziós (sztereó) tömböket dolgozza fel másodpercenként többször.

1. Effektív hangerőszint (`calculate_rms` és `calculate_stereo_rms`): A jel energiájának kiszámításához a Root Mean Square (RMS) képletet alkalmazom. A metódus a bejövő lebegőpontos hangminták négyzetes átlagának gyökét veszi:

$$\text{RMS} = \sqrt{\frac{1}{N} \sum_{i=1}^N x_i^2} \quad (3.1)$$

Ezt sztereó jelnél a bal és jobb csatornára külön is kiszámítom, hogy kivezérlésjelzőt (VU Meter) tudjak megjeleníteni a kliens oldalon.

2. Zeneszám kezdetének detektálása (`check_music_start`): Annak megállapítására, hogy a tű a lemezre ért-e, a rendszer figyeli az RMS értéket. Ha ez az érték egy meghatározott küszöb (alapértelmezetten 0.01) felett marad egy megadott időtartamig (pl. 0.5 másodperc), a szoftver regisztrálja a lejátszás kezdetét. Ugyanezen elv fordítottjával működik a `check_silence_start` metódus, amely a zeneszámok közötti csendes sávokat ismeri fel.

3. Kattogások detektálása (`detect_clicks`): A bakelitlemezek felületi sérülései hirtelen, magas amplitúdójú tranziensekként (pattanásként) jelentkeznek. A metódus először monósítja a jelet, majd kiszámítja az első deriváltját (két egymást követő minta különbségét), ami felüláteresztő szűrőként funkcionál:

$$d[n] = x[n] - x[n - 1] \quad (3.2)$$

Ezután kiszámítom az abszolút deriváltak átlagát (μ) és szórását (σ). Egy dinamikus küszöbértéket (T) állapítok meg, amely egy felhasználó által állítható érzékenységi

szorzótól (k) függ:

$$T = \mu + k\sigma \quad (3.3)$$

Minden olyan minta, amely meghaladja ezt a küszöböt, statisztikai kiugró értéknek számít, és növeli a kattogásszámlálót.

4. Sztereó egyensúly (`get_channel_balance`): A lemezjátszó anti-skating beállításának diagnosztikájához a jobb (R) és bal (L) csatorna RMS értékeiből kiszámítom a sztereó egyensúlyt:

$$\text{Balance} = \frac{R - L}{R + L} \quad (3.4)$$

Az eredmény egy -1 és $+1$ közötti érték, ahol a 0 a tökéletes közepet jelenti.

5. Mechanikai motorrezgés (`measure_rumble`): A lemezjátszó motorjának nem kívánt rezgéseit Fast Fourier Transzformációval (FFT) mérem. A módszer frekvenciatartományba transzformálja a jelet, majd kizárólag a 10 Hz és 50 Hz közötti spektrális energiát összegzi:

$$\text{Rumble} = \sum_{f=10}^{50} |X(f)| \quad (3.5)$$

6. Magas frekvenciás torzítás (`detect_sibilance`): A sibilance (sziszegés) méréséhez a SciPy `signal.butter` módszerével egy sávszűrőt (bandpass filter) hozok létre, amely 5000 Hz és 10000 Hz között engedi át a jelet. A szűrt jel RMS értékét elosztom a teljes jel RMS értékével. Ha ez az arány kirívóan magas, az a tűkövetési hibájára (tracking distortion) utal.

A fenti algoritmusok stabilitását a fizikai tesztek során finomhangolt határértékek biztosítják. A 3.2. táblázat foglalja össze az audioelemzés során alkalmazott legfontosabb frekvenciatartományokat és alapértelmezett küszöbértékeket.

Metrika / Diagnosztika	Frekvenciatartomány	Alapértelmezett küszöbérték / Képlet
Zene kezdetének detektálása	Teljes spektrum	$RMS > 0.01$ (legalább 0.5 mp hosszan)
Lejátszás végének (csend) detektálása	Teljes spektrum	$RMS < 0.01$ (legalább 2.0 mp hosszan)
Kattogás (Click) detektálás	Magas frekvenciás transziensek (Derivált jel)	Dinamikus küszöb: $T = \mu + (15 \times \sigma)$
Mechanikai rezgés (Rumble)	10 Hz – 50 Hz	N/A (A sávhoz tartozó FFT energia összege)
Sziszegő torzítás (Sibilance)	5 kHz – 10 kHz	A szűrt jel RMS aránya a teljes jelhez viszonyítva > 0.2 (20%)

3.2. táblázat. A digitális jelfeldolgozás során alkalmazott frekvenciasávok és határértékek.

3.3.2. Zenefelismerés (Recognition) – `recognition_service.py`

A felismerő modul fejlesztését egy kiterjedt kutatási fázis előzte meg. Három szolgáltatást vizsgáltam: az ACRCLOUD-ot, az AcoustID-t és a Shazamio-t. Az AcoustID nyílt forráskódú, de a tesztek során kiderült, hogy tiszta audiojelet igényel, így a bakelitre jellemző felületi zajok miatt egyetlen lemezt sem ismert fel. Az ACRCLOUD pontos volt, de az ingyenes próbaidőszak korlátozásai miatt ipari használatra tervezték.

A végső megoldás a Shazamio [4] könyvtár lett, amely a Shazam aszinkron, visszafejtett (reverse engineered) API-ja. Az `identify_audio` metódus aszinkron módon elküld egy 8 másodperces hangmintát a Shazamnak. Ha a Shazam felismeri, visszaadja a szám címét, előadóját és egy Apple Music azonosítót. Ha a Shazam valamilyen oknál fogva hibára fut (fallback), a metódus másodlagos szolgáltatásként az ACRCLOUD SDK-t használja a felismeréshez.

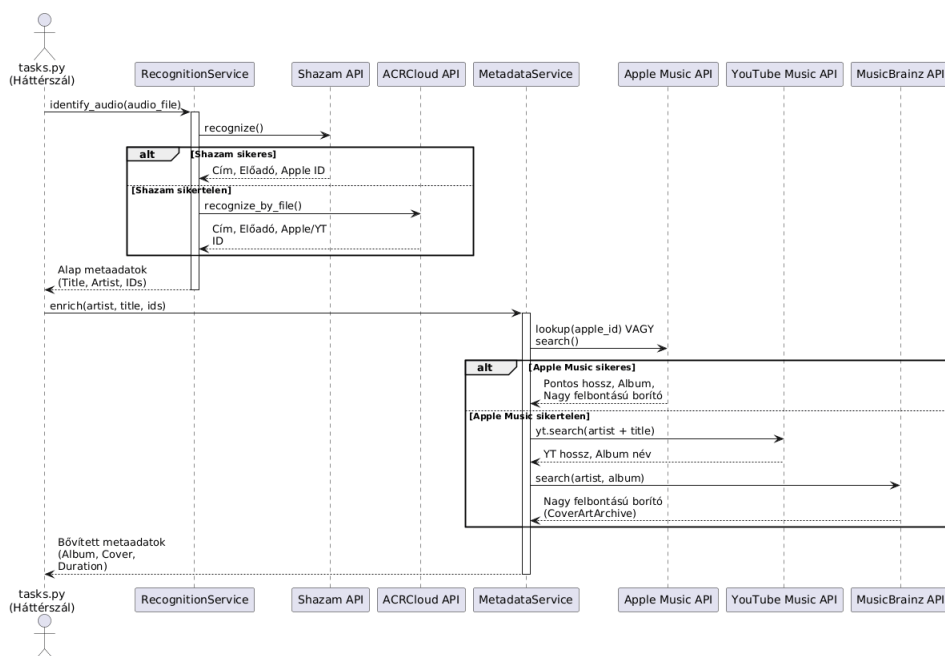
3.3.3. Metaadat-vízesés (Waterfall) – `metadata_service.py`

A pusztán cím és előadó nem elegendő a teljes felhasználói élményhez. Az `enrich` metódus egy szigorú prioritási sorrendet, úgynevezett vízesés (waterfall) modellt követ a metaadatok beszerzésére.

1. **Apple Music API (`fetch_apple`):** Elsődlegesen a Shazam által visszaadott Apple azonosító alapján lekérdezem az iTunes API-t. Ha sikeres, megkapom a

pontos sávhosszúságot (milliszekundumban), az album hivatalos nevét és egy nagy felbontású (600x600 px) borítóképet.

- YouTube Music API (fetch_youtube):** Ha az Apple Music nem ismeri a dalt, a ytmusicapi segítségével végzek szöveges keresést. Mivel a YouTube alapértelmezetten kis felbontású videó-bélyegképeket ad vissza, az URL manipulálásával (w120-h120 cseréje w1000-h1000-re) kényszerítem ki a jó minőségű képet.
- MusicBrainz Fallback (fetch_musicbrainz_cover):** Ha a YouTube Music sem ad megfelelő borítót, harmadik lépcsőként a CoverArtArchive REST API-ját hívom meg az előadó és az album neve alapján.



3.3. ábra. A Fallback és Metaadat-vízesés folyamata (UML Sequence Diagram).

3.3.4. Háttérfolyamatok és Szálkezelés – tasks.py

A valós idejű működés magját az `audio_processing_thread` háttérrel adja. Ez a szál egy végtelen ciklusban, 4096 mintás blokkokban (chunk) olvassa a hangkártya bemenetét. Minden blokkon lefuttatja a korábban említett DSP metódusokat (RMS, click, rumble, sibilance).

Az I/O műveletek optimalizálása érdekében a szál egy `buffer_seconds` memóriaváltozóban halmozza fel a lejátszott időt, és csak 10 másodpercenként írja az

adatbázisba (`Cartridge` kopás mentése). Minden iteráció végén a szál a `SocketIO` csatornán keresztül egy `stats_update` eseményben kisugározza az élő adatokat a kliensnek.

Amikor a DSP modul megállapítja egy új dal kezdetét, a rendszer elindítja az `identify_and_save` metódust egy külön szálon. Ez:

1. Rögzít 8 másodpercnyi hangot (`AudioCapture`).
2. Meghívja a `RecognitionService`-t.
3. Meghívja a `MetadataService`-t.
4. Hálózati lekérdezést indít a `lrclib.net` API felé, hogy letöltse az időbélyeggel ellátott dalszöveget.
5. Elmenti a sikeres találatot a `TrackHistory` adatbázis táblába.

Végül, a lejátszás során egy figyelő logika vizsgálja, hogy a sáv eltelt ideje elérte-e az 50%-ot (vagy a 4 percet). Ha igen, a `scrobble_track_async` háttérfolyamat aszinkron módon regisztrálja a lejátszást a felhasználó Last.fm fiókjában.

3.4. Tesztelés és Értékelés

3.4.1. Automatizált Egységtesztelés (Unit Testing)

A rendszer stabilitásának és hosszú távú karbantarthatóságának biztosítása érdekében a Python beépített `unittest` keretrendszerével átfogó automatizált egységteszteket (Unit Tests) írtam. Mivel az alkalmazás fizikai hardverekkel (lemezjátszó, hangkártya) és külső webes API-kkal is kommunikál, a tesztelés során kiemelt feladat volt ezeknek a külső függőségeknek a szimulálása (Mocking) és izolálása. A tesztek logikailag három fő kategóriába sorolhatók:

1. Adatbázis és API végpontok tesztelése (`test_api.py`): A REST API és az adatbázis-modellek teszteléséhez a Flask keretrendszer teszt-kliensét (`test_client`) használtam. Annak érdekében, hogy a tesztek futtatása ne tegye tönkre a valós, éles adatbázisban tárolt felhasználói adatokat, a tesztkörnyezet egy memóriában futó SQLite adatbázist (`sqlite:///memory:`) inicializál. Ez a memóriabázis a tesztek lefutása után azonnal megsemmisül. A tesztek során szimuláltam a kliens bejelentkezését (`POST /auth/login`), ellenőriztem a jogosultságkezelést (nem

hitelesített kérések elutasítása 401-es hibakóddal), valamint teszteltem a hardveres beállítások és a hangszedő limitjeinek sikeres módosítását.

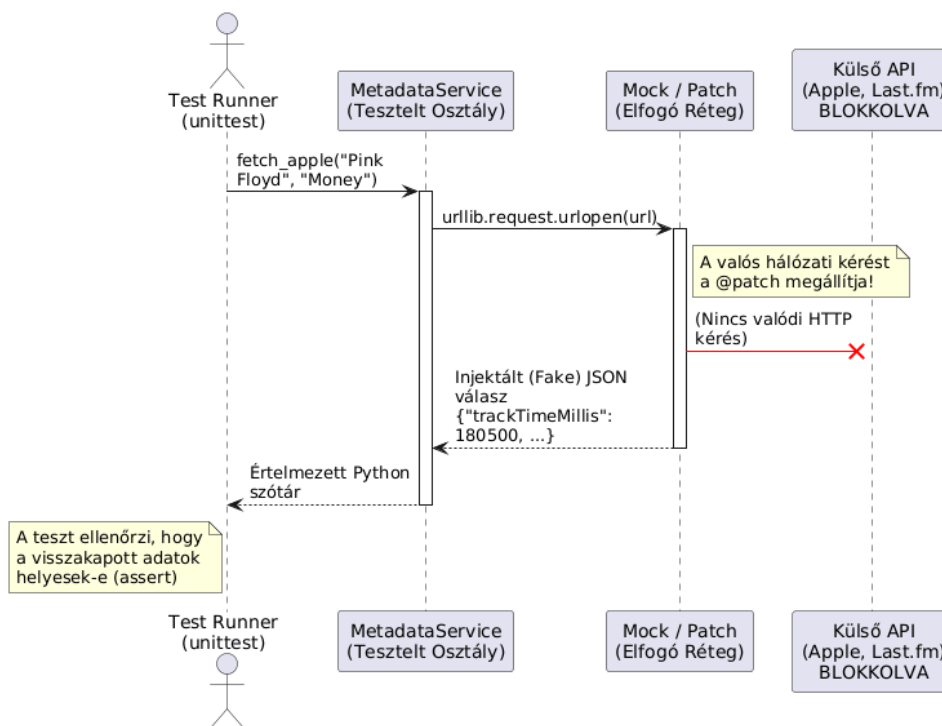
2. Digitális Jelfeldolgozás szimulálása (test_audio_dsp.py): A DSP algoritmusok pontosságának igazolásához nem volt szükség valós fizikai hanglemezek lejátszására a tesztkörnyezetben. Ehelyett a NumPy könyvtár segítségével szintetikus (mesterségesen generált) matematikai tömböket hoztam létre, amelyek egy tökéletes hangkártya kimenetét imitálják:

- **Csend és Konstans jelek:** Egy teljesen nullákkal feltöltött tömbbel (`np.zeros`) igazoltam, hogy a rendszer pontosan 0.0 RMS értéket ad vissza, egy fix értékekkel feltöltött tömbbel pedig a matematikai átlagolás helyességét ellenőriztem.
- **Kattogás (Click) szimuláció:** Egy csendes tömb közepébe egyetlen masszív értéket (tüskét) injektáltam (`audio_chunk[2000] = [1.0, 1.0]`). Az algoritmus sikeresen megtalálta a statisztikai kiugrást.
- **Frekvencia-generálás (Rumble és Sibilance):** A frekvenciaszűrők teszteléséhez szinuszgörbéket generáltam a matematikai képlet ($\sin(2\pi \cdot f \cdot t)$) alapján. Egy tiszta 30 Hz-es szinuszhullámot generálva a `measure_rumble` metódus vártan magas motorrezgés-értéket mutatott, míg egy 8000 Hz-es (8 kHz) szinuszhullám sikeresen triggerelte a `detect_sibilance` (sziszegés) mérőeszközt, igazolva a Butterworth sávszűrő hibátlan működését.

3. Külső szolgáltatások izolálása Mocking-gal (test_services.py): A harmadik féltől származó API-kat (Apple Music, YouTube Music, Last.fm) hívó szolgáltatások tesztelése speciális kihívást jelentett. Nem engedhető meg, hogy a tesztek futtatása valós hálózati forgalmat generáljon (ami terhelné az API-kat, vagy internet hiányában a tesztek elbukását okozná). Ezt a problémát a Python `unittest.mock.patch` dekorátorával oldottam meg.

A szimuláció (Mocking) során a kód elhiteti a szolgáltatással, hogy a hálózati kérés lefutott. Amikor például a rendszer meghívja az `urllib.request.urlopen` metódust, a `@patch` megállítja a kérést, és egy előre megírt, mesterséges (fake) JSON választ ad vissza (lásd 3.4. ábra). Ezzel a módszerrel sikeresen leteszteltem:

- A **Waterfall (Vízesés) metaadat-logikát**, megfigyelve, hogy az algoritmus valóban továbblép-e a YouTube keresésre, ha az Apple Music szimulált válasza üres.
- A dalszövegek (LRC) reguláris kifejezésekkel (Regex) történő időbélyegfeldolgozását ([MM:SS.xx]).
- A Last.fm scrobbling folyamatot, ahol a hálózati réteget teljes egészében szimulált MagicMock objektumokkal helyettesítettem, bizonyítva, hogy a belső logika az API szervereinek elérhetetlensége esetén is biztonságosan, összeomlás nélkül lefut.



3.4. ábra. UML Sequence Diagram a külső API-k @patch alapú szimulálásáról (Mocking).

3.4.2. Frontend renderelés, Vizuális Megjelenítés és Témázás

A felhasználói felület (UI) megjelenítése a letisztult elrendezés mellett a személyre szabhatóságra fókuszál. Az egyedi stíluslapok (`themes.css`, `dashboard.css`) segítségével a rendszer vizuálisan is alkalmazkodik a felhasználó preferenciáihoz.

Dinamikus Témaváltás (Modern vs. Retro): Az alkalmazás megjelenését a gyökér (`:root`) elemen definiált CSS változók vezérlik, amelyeket a HTML DOM-on lévő `data-theme` és `data-style` attribútumok manipulálnak.

- A **Modern** stílus (`data-style="modern"`) lekerekített sarkokat (`-modern-radius`), finom árnyékokat és lágy átmeneteket használ a ki-vezérlésjelzőknél (függőleges LED sávok).
- A **Retro** stílus (`data-style="retro"`) ezzel szemben a klasszikus Windows 95/98 esztétikát idézi: éles sarkok (`0px border-radius`), `inset/outset` dobozárnnyékok (bevel effect), klasszikus analóg mutatós VU méterek, és monospace (Courier New) tipográfia.

A Bakelitlemez Renderelése és Animációja: A forgó lemez vizuális illúzióját tiszta CSS megoldások adják, elkerülve a memóriaintenzív 3D renderelést.

- **Textúrák és Színek:** A lemez alapját több mint 30 választható CSS osztály adja. Az egyszerű színek (pl. `.v-ruby`) átlátszósággal (`opacity: 0.9`) operálnak. Az összetettebb, több színű lemezek matematikai színátmenetekkel készülnek, például a *Rasta Split* lemez egy `conic-gradient(#e20613 0 120deg, #f9e514 120deg 240deg...)` kód segítségével osztja a kört három egyenlő cikkelyre. Más komplex mintákat (pl. *Galaxy Splash*) nagy felbontású ismétlődő háttérképek (`background: url(...)`) adnak.
- **A Barázdák illúziója:** A hanglemez textúrájára egy `::after` pseudo-element borul, amely egy `repeating-radial-gradient` segítségével koncentrikus, finoman átlátszó köröket rajzol, tökéletesen imitálva a fizikai barázdákat. Mivel ezen az elemen `pointer-events: none` van, a felhasználó gond nélkül tud kattintani a lemezre.
- **Forgás (Rotation):** Amikor az audio elemző zenét detektál, a Vue keretrendszer dinamikusan rárakja a DOM elemre a `spinning` osztályt. Ez egy végtelenített CSS `@keyframes spin` animációt indít el, amely 4 másodperc alatt forgatja körbe a 360 fokot (`transform: rotate(360deg)`).

3.4.3. Kliens-oldali Logika és Állapotkezelés (useDashboard.js)

A Főképernyő (Dashboard) mögötti reaktív logikát a Vue 3 Composition API-ra épülő `useDashboard` és `useVinylInteractions` modulok kezelik. A legfontosabb folyamatok és metódusok a következők:

1. Socket.IO kommunikáció és adatszinkronizáció:

- **onMounted:** A komponens betöltésekor inicializálja a WebSocket kapcsolatot a backenddel. Három fő eseményre (event) iratkozik fel: `stats_update`, `status_change`, és `track_identified`.
- **stats_update** figyelő: Másodpercenként többször lefut. Frissíti a Vue reaktív (`ref`) változóit: az RMS értékeket (L/R kivezérlés), a lejátszási időt (`trackTime`), a mechanikai zajokat (`currentRumble`, `currentSibilance`), és betölti az aktuális kattogásokat a `clickHistory` tömbbe az idővonal megrajzolásához.
- **status_change** figyelő: Kezeli a felület "Keresés folyamatban" (Identifying...) állapotát, blokkolva a felhasználói interakciókat a keresés alatt.
- **track_identified** figyelő: Sikeres találat esetén betölti a borítóképet, és azonnal meghívja a dalszöveg-feldolgozó rutint, majd a képernyőt a megfelelő sorhoz görgeti.
- **onUnmounted:** Biztonságosan lezárja a socket kapcsolatot (`socket.disconnect()`), amikor a felhasználó elhagyja az oldalt, megelőzve a memóriaszivárgást.

2. Dalszöveg (Lyrics) feldolgozása és szinkronizációja:

- **parseLRC:** Reguláris kifejezések (Regex: `\[\d{2}:\d{2}(\.\d+)?\]`) segítségével soronként végigmegy a backendtől kapott LRC formátumú stringen. Kinyeri a perceket és másodperceket, lebegőpontos másodperccé alakítja őket (`time`), és egy objektumtömböt ad vissza. Ha a dalszöveg nem szinkronizált, a `time` értékét -1-re állítja.
- **activeLyricIndex (Computed):** Egy számított tulajdonság, amely folyamatosan figyeli a `trackTime` változását. Visszafelé iterál a `parsedLyrics` tömbön,

és visszaadja annak a sornak az indexét, amelynek az időbélyege kisebb vagy egyenlő az aktuális lejátszási idővel.

- **scrollToActive:** Amikor az `activeLyricIndex` megváltozik, ez az aszinkron metódus a Vue `nextTick()` függvényét használja, hogy megvárja a DOM frissülését, majd a beépített `scrollIntoView` paranccsal finoman (`behavior: 'smooth'`) a képernyő közepére görgeti az aktív sort.
- **adjustSync:** Manuális csúszáskorrekció. A felhasználó gombnyomására (pl. +0.25 mp) kiszámolja az eltolást, hozzáadja a helyi `trackTime`-hoz, és a `adjust_track_time` eseménnyel visszaküldi a backendnek.
- **resetSync:** Törli a manuális eltolásokat a backend oldalon.
- **handleUserScroll:** Ha a felhasználó belegörget a dalszövegbe (`@wheel`, `@touchstart`), azonnal kikapcsolja az automatikus követést (`isAutoScrollEnabled = false`).
- **resyncLyrics:** Újra aktiválja az automatikus görgetést, és azonnal a helyes sorra ugrik.

3. Felhasználói interakciók és Vizuális funkciók:

- **triggerManualDetect:** Kikényszerít egy azonnali Shazam azonosítást a backendtől (`manual_detect` emitálása).
- **setVinylColor:** A jobb gombos menüből kiválasztott szín osztálynevét (pl. `v-ruby`) elmenti a `currentTrack` objektumba, és elküldi az adatbázisba.
- **openContextMenu / closeContextMenu:** Kiszámolja az egérkurzor pontos X és Y koordinátáit (`e.clientX`, `e.clientY`), figyelembe véve az ablak szélességét, hogy a lemez-stílus választó menü ne lógjon ki a képernyőről.
- **handleContainerClick / triggerPopcorn:** Egy rejtett "Easter Egg". Ha a felhasználó Modern témában ötször gyorsan rákattint a lemezre (egy 400 ms-os `setTimeout` ablakon belül), a rendszer dinamikusan generál 30 darab tűz (*Fire*) emoji-t véletlenszerű méretezéssel (`Math.random()`) és röppályával, majd 1.5 másodperc múlva automatikusan eltávolítja őket a DOM-ból.
- **formatTime:** Másodperceket alakít át szabványos `MM:SS` (perc:másodperc) string formátumra, a `padStart` string metódus alkalmazásával.

3.4.4. Profilkezelés és Hardver Konfiguráció (useProfile.js)

A felhasználói beállítások és integrációk kezelését a `Profile.vue` és annak logikája biztosítja, REST API hívások (Axios / Fetch) segítségével.

1. Adatbetöltés és Mentés:

- `loadData`: A profil megnyitásakor a `Promise.all()` segítségével párhuzamosan kéri le a backendtől a felhasználói statisztikákat (történet, hangszedők) és a csatlakoztatott fizikai audió eszközök (hangkártyák) listáját.
- `saveSettings`: Az audió bemenet (`audio_device_id`) és a DSP csúszkák (RMS threshold, Click sensitivity) értékeit küldi el a backendnek. A `isSaving` flag közben letiltja a mentés gombot a dupla kattintások elkerülése végett.

2. Hangszedő (Cartridge) CRUD műveletek:

- `addNewCartridge`: Egy új JSON objektumot (`newCartData`) küld a servernek, tartalmazva a tű nevét és a gyártó által javasolt maximális üzemórát, majd frissíti a lokális listát.
- `toggleCartridge`: Harmonika (accordion) stílusban nyitja/zárja a listában szereplő tűk részletes beállításait az `expandedCartId` változó manipulálásával.
- `activateCartridge`: A kiválasztott tűt állítja be alapértelmezettnek.
- `updateCartridgeLimit`: Módosítja a már regisztrált tű maximális élettartamát.
- `resetCartridge`: Egy megerősítő felugró ablak (`confirm`) után nullázza a kiválasztott tű eddig mért kopási idejét.
- `deleteCartridge`: Véglegesen törli a hardverelemet az adatbázisból.

3. Last.fm Integráció (OAuth folyamat):

- `connectLastFm`: Összeállítja a Last.fm hitelesítési URL-jét a fejlesztői API kulccsal (`LASTFM_API_KEY`) és a visszatérési címmel (`callbackUrl`), majd át-irányítja a böngészőt.
- `onMounted` (Token figyelés): Ha az URL-ben szerepel a `?token=` paraméter (a Last.fm-től való sikeres visszatérés után), a frontend ezt automatikusan elfogja, elküldi a backendnek véglegesítésre (`api.connectLastFm(token)`), majd

a `window.history.replaceState` paranccsal letisztítja az URL-t a biztonság érdekében.

- `disconnectLastFm`: Megszünteti a kapcsolatot és törli a Session Key-t az adatbázisból.

3.4.5. Manuális és integrációs tesztelés

A rendszert különféle fizikai bakelitlemezekkel (eltérő műfajok és felületi állapotok) értékeltük. Ismert fizikai sérülésekkel rendelkező lemezeket játszottunk le annak megerősítésére, hogy a pattogás detektáló (click detection) logika nem ad fals pozitív találatokat agresszív dob ütéseknel. A sibilance és a rumble mérőket szándékos követési torzítással, illetve motorrezgéssel teszteltük az FFT és a szűrők matematikai érvényességét.

3.4.6. Teljesítmény-optimalizáció

A kezdeti tesztelés feltárta, hogy a hangszedő statisztikáinak másodpercenkénti írása súlyos I/O szűk keresztmetszeteket okozott. Ezt az SQLite Write-Ahead Logging (WAL) módjának engedélyezésével és egy memóriabufferelő rendszer implementálásával oldottuk meg. A rendszer az értékeket a memóriában halmozza fel, és csak 10 másodpercenként írja ki a fizikai tárhelyre, ami drasztikusan csökkentette a CPU terhelését egy Raspberry Pi környezetben.

3.5. Mesterséges intelligencia alkalmazása a fejlesztés során

A projekt megvalósítása során modern nagynyelvi modellek (LLM) alkalmazása történt, amelyek elsődlegesen segítő eszközként, a fejlesztési folyamat gyorsítása és optimalizálása érdekében szolgáltak. Az AI használata a következő területeken történt:

- **Koncepciók elsajátítása és kutatás:** Ismeretlen technológiai területek (például a digitális jelfeldolgozás specifikus matematikai műveletei vagy komplex API integrációk) esetén az AI-t mint gyors referenciát használtam. A kapott

példakódokat nem implementáltam közvetlenül, hanem alapvető iránymutatónak tekintettem őket, amelyeket egy egyedi tervezési folyamat után adaptáltam a projekt specifikus igényeihez.

- **Ismétlődő kódstruktúrák (boilerplate) generálása:** A fejlesztés során felmerülő repetitív feladatok, mint például a frontend és backend közötti alapvető adatcsere logikája vagy a standard API végpontok vázlata, generálása során alkalmaztam AI segítségét. Ezek a kódrészletek egy alapvázlat szolgáltatottak, amelyek végleges formáját manuális átírással és finomhangolással értem el.
- **Hibakeresés és optimalizálás:** A debugolási folyamat során az AI-t kódrészletek elemzésére és potenciális hibaforrások gyorsabb azonosítására használtam. Mivel az AI nem ismerte a teljes projekt architektúráját, a javasolt megoldásokat kritikus szemlélettel szűrtem, és csak a projekt kontextusába illeszthető megoldásokat integráltam a végleges kódbázisba.

Külön kiemelném, hogy a rendszer architektúrájának tervezése, a specifikus matematikai logikák illesztése, valamint a végleges szoftveres implementáció teljes egészében a szerző munkájának eredménye. Az AI használata kizárólag a fejlesztési ciklus hatékonyságának növelésére és a tanulási folyamat gyorsítására szolgált.

4. fejezet

Összegzés és jövőbeli fejlesztések

4.1. Az elért eredmények értékelése

A Smart Turntable Assistant sikeresen eléri elsődleges célját, az analóg bakelitlemezhallgatási élmény modernizálását. A nyers elektromos audiojel passzív monitorozásával a rendszer áthidalja a fizikai média és a digitális ökoszisztéma közötti szakadékot. Az alkalmazás közel 80% pontossággal azonosítja a lejátszott sávokat, a hallgatási munkamenetet nagy felbontású borítóképekkel és szinkronizált dalszövegekkel gazdagítja, és zökkenőmentesen naplózza a hallgatási előzményeket a külső közösségi zenei platformokra.

Továbbá, az audiofil szintű diagnosztikai eszközök beépítése valós idejű vizuális visszajelzést biztosít a felhasználóknak a tű és a lemezek fizikai állapotáról. A rendkívül moduláris, szétválasztott (decoupled) architektúra biztosítja, hogy a mögöttes Python backend és a reaktív webes interfész hatékonyan működjön egy olyan professzionális és robusztus szoftveres megoldásban, amely tiszteletben tartja a hagyományos hanglemez élményt, miközben modern digitális kényelmet nyújt.

4.2. Jövőbeli fejlesztési lehetőségek

Bár a jelenlegi implementáció teljesíti az összes kezdeti követelményt, számos lehetőség kínálkozik a jövőbeli továbbfejlesztésre. A legjelentősebb fejlődés saját neuron hálók használatával való sibilance detection és két zeneszám közti különbség (tempó és hangszín) detektálásával lenne elérhető.

Ezenfelül a diagnosztikai csomag kibővíthető lenne egy automatizált lemezjátszó-kalibrációs móddal is. Egy lemez lejátszásával ki tudja választani a felhasználó, hogy hol megy már a zene, ezzel a music detection algoritmus érzékenységét egyből be tudja állítani egy elfogadható állapotra, minimalizálva a manuális konfigurációt.

Ábrák jegyzéke

2.1. A Főképernyő lejátszó panelje aktív zenehallgatás közben.	10
2.2. Szinkronizált dalszöveg megjelenítése és finomhangoló kezelőszervei. .	11
2.3. Hardveres diagnosztika, kivezérlésjelzők és a pattogások idővonala. . .	13
2.4. Profilkezelés, hangszedő konfiguráció és hardveres érzékenység beállítása.	14
3.1. Entity-Relationship Diagram az adatbázis sémájáról.	17
3.2. UML Sequence Diagram a többszálú adatfeldolgozásról.	18
3.3. A Fallback és Metaadat-vízesés folyamata (UML Sequence Diagram). .	22
3.4. UML Sequence Diagram a külső API-k @patch alapú szimulálásáról (Mocking).	25

Ábrák jegyzéke

Táblázatok jegyzéke

3.1. A rendszer alapvető REST API végpontjai és azok funkciói.	16
3.2. A digitális jelfeldolgozás során alkalmazott frekvenciasávok és határ- értékek.	21

Algoritmusjegyzék

Forráskódjegyzék